Vehicle Rental Management System

Java OOP + Exception Handling | 2-Hour Advanced Lab

A compact but concept-dense practical problem designed to test core Java Object-Oriented Programming and Exception Handling concepts in one implementation.

1. Problem Statement

Build a Vehicle Rental Management System in Java. The system should manage different types of vehicles, customers, rentals and payments. The implementation must demonstrate abstraction, inheritance, encapsulation, interfaces, polymorphism, method overloading, constructor overloading, static and final members, association, arrays of objects, and exception handling.

Functional Requirements

1. Create an abstract Vehicle class with common vehicle information.
2. Create Car, Bike and ElectricCar subclasses using inheritance.
3. Create a Customer class with overloaded constructors.
4. Create a Payment interface and implement it in Rental.
5. Create a Rental class connecting a Customer and Vehicle.
6. Calculate rental charges using overloaded methods.
7. Use runtime polymorphism through a Vehicle[] array.
8. Maintain static counters for vehicles and rentals.
9. Use a final GST constant.
10. Validate invalid vehicle/customer/rental data.
11. Create and use custom checked exceptions.
12. Demonstrate try, catch, finally, throw and throws.
13. Do not use Collections. Use arrays only.

2. Class Requirements

Class / Interface	Important Requirements
Vehicle (abstract)	private vehicleId, brand, rentPerDay, available; static totalVehicles; constructor; abstract calculateRent(); final displayBas
Car	extends Vehicle; numberOfSeats; override calculateRent() and displayDetails().
Bike	extends Vehicle; engineCC; override calculateRent() and displayDetails().
ElectricCar	extends Vehicle; batteryCapacity; override calculateRent() and displayDetails().
Customer	private customerId, name, phone; overloaded constructors using this().
Payment	pay(double amount).
Rental	Customer + Vehicle association; static totalRentals; overloaded calculateTotal methods; implements Payment.

3. Exception Requirements

InvalidVehicleException: vehicleId <= 0, empty brand, or rentPerDay <= 0.

InvalidCustomerException: customerId <= 0, empty name, or invalid phone.

VehicleNotAvailableException: Attempting to rent a vehicle whose available flag is false.

The rental calculation method should declare VehicleNotAvailableException with throws and should use throw when the vehicle is unavailable. The main method must handle the exceptions using try-catch-finally.

4. OOP Concepts to Demonstrate

Classes and Objects • Encapsulation • Constructors and Constructor Overloading • this and super • Inheritance • Abstract Class and Abstract Method • Interface • Method Overloading • Method Overriding • Runtime Polymorphism • static • final • Association • Arrays of Objects • try-catch-finally • throw • throws • Custom Exceptions

Complete Solution

Single-file solution: VehicleRentalSystem.java

```
interface Payment {
    void pay(double amount);
}

class InvalidVehicleException extends Exception {
    public InvalidVehicleException(String message) {
        super(message);
    }
}

class InvalidCustomerException extends Exception {
    public InvalidCustomerException(String message) {
        super(message);
    }
}

class VehicleNotAvailableException extends Exception {
    public VehicleNotAvailableException(String message) {
        super(message);
    }
}

abstract class Vehicle {
    private int vehicleId;
    private String brand;
    private double rentPerDay;
    private boolean available;

    private static int totalVehicles = 0;

    public Vehicle(int vehicleId, String brand, double rentPerDay)
            throws InvalidVehicleException {

        if (vehicleId <= 0)
            throw new InvalidVehicleException("Vehicle ID must be positive.");

        if (brand == null || brand.trim().isEmpty())
            throw new InvalidVehicleException("Brand cannot be empty.");

        if (rentPerDay <= 0)
            throw new InvalidVehicleException("Rent per day must be positive.");

        this.vehicleId = vehicleId;
        this.brand = brand;
        this.rentPerDay = rentPerDay;
        this.available = true;
        totalVehicles++;
    }

    public int getVehicleId() {
        return vehicleId;
    }

    public String getBrand() {
        return brand;
    }

    public double getRentPerDay() {
        return rentPerDay;
    }

    public boolean isAvailable() {
        return available;
    }

    public void setAvailable(boolean available) {
        this.available = available;
    }

    public abstract double calculateRent(int days);

    public final void displayBasicDetails() {
        System.out.println("ID: " + vehicleId +
                ", Brand: " + brand +
                ", Rent/Day: " + rentPerDay +
                ", Available: " + available);
    }

    public static void showVehicleCount() {
        System.out.println("Total Vehicles: " + totalVehicles);
    }
}
```

```
class Car extends Vehicle {
    private int numberOfSeats;

    public Car(int id, String brand, double rent, int seats)
            throws InvalidVehicleException {

        super(id, brand, rent);

        if (seats <= 0)
            throw new InvalidVehicleException("Seats must be positive.");

        this.numberOfSeats = seats;
    }

    @Override
    public double calculateRent(int days) {
        return getRentPerDay() * days + 200;
    }

    public void displayDetails() {
        displayBasicDetails();
        System.out.println("Type: Car, Seats: " + numberOfSeats);
    }
}

class Bike extends Vehicle {
    private int engineCC;

    public Bike(int id, String brand, double rent, int engineCC)
            throws InvalidVehicleException {

        super(id, brand, rent);

        if (engineCC <= 0)
            throw new InvalidVehicleException("Engine CC must be positive.");

        this.engineCC = engineCC;
    }

    @Override
    public double calculateRent(int days) {
        return getRentPerDay() * days;
    }

    public void displayDetails() {
        displayBasicDetails();
        System.out.println("Type: Bike, Engine: " + engineCC + " CC");
    }
}

class ElectricCar extends Vehicle {
    private double batteryCapacity;

    public ElectricCar(int id, String brand, double rent, double batteryCapacity)
            throws InvalidVehicleException {

        super(id, brand, rent);

        if (batteryCapacity <= 0)
            throw new InvalidVehicleException(
                    "Battery capacity must be positive.");

        this.batteryCapacity = batteryCapacity;
    }

    @Override
    public double calculateRent(int days) {
        return getRentPerDay() * days + 100;
    }

    public void displayDetails() {
        displayBasicDetails();
        System.out.println("Type: Electric Car, Battery: "
                + batteryCapacity + " kWh");
    }
}

class Customer {
    private int customerId;
    private String name;
    private String phone;

    public Customer() {
        this(0, "Unknown", "0000000000");
    }

    public Customer(int customerId, String name)
            throws InvalidCustomerException {
        this(customerId, name, "0000000000");
    }
```

```
    public Customer(int customerId, String name, String phone)
            throws InvalidCustomerException {

        if (customerId <= 0)
            throw new InvalidCustomerException(
                    "Customer ID must be positive.");

        if (name == null || name.trim().isEmpty())
            throw new InvalidCustomerException(
                    "Customer name cannot be empty.");

        if (phone == null || !phone.matches("\\d{10}"))
            throw new InvalidCustomerException(
                    "Phone must contain exactly 10 digits.");

        this.customerId = customerId;
        this.name = name;
        this.phone = phone;
    }

    public String getName() {
        return name;
    }

    public String toString() {
        return "Customer ID: " + customerId +
                ", Name: " + name +
                ", Phone: " + phone;
    }
}

class Rental implements Payment {
    private Customer customer;
    private Vehicle vehicle;

    private static int totalRentals = 0;
    private final double GST = 0.18;

    public Rental(Customer customer, Vehicle vehicle)
            throws VehicleNotAvailableException {

        if (!vehicle.isAvailable())
            throw new VehicleNotAvailableException(
                    "Vehicle is currently unavailable.");

        this.customer = customer;
        this.vehicle = vehicle;
        vehicle.setAvailable(false);
        totalRentals++;
    }

    // Method overloading - version 1
    public double calculateTotal() throws VehicleNotAvailableException {
        return calculateTotal(1);
    }

    // Method overloading - version 2
    public double calculateTotal(int days)
            throws VehicleNotAvailableException {

        if (!vehicle.isAvailable() && days <= 0)
            throw new VehicleNotAvailableException(
                    "Invalid rental duration.");

        if (days <= 0)
            throw new IllegalArgumentException(
                    "Days must be greater than zero.");

        double base = vehicle.calculateRent(days);
        return base + (base * GST);
    }

    // Method overloading - version 3
    public double calculateTotal(int days, double discount)
            throws VehicleNotAvailableException {

        if (discount < 0 || discount > 100)
            throw new IllegalArgumentException(
                    "Discount must be between 0 and 100.");

        double base = vehicle.calculateRent(days);
        double discounted = base - (base * discount / 100);
        return discounted + (discounted * GST);
    }

    @Override
    public void pay(double amount) {
        System.out.println("Payment successful: Rs. " + amount);
        vehicle.setAvailable(true);
    }
```

```
    public static void showRentalCount() {
        System.out.println("Total Rentals: " + totalRentals);
    }
}

public class VehicleRentalSystem {

    public static void main(String[] args) {

        try {
            // Constructor + inheritance + exception validation
            Vehicle[] vehicles = new Vehicle[3];

            vehicles[0] = new Car(101, "Toyota", 1500, 5);
            vehicles[1] = new Bike(102, "Honda", 700, 160);
            vehicles[2] = new ElectricCar(103, "Tata", 1800, 40);

            // Runtime polymorphism
            System.out.println("===== VEHICLES =====");

            for (Vehicle v : vehicles) {
                v.displayBasicDetails();
                System.out.println(
                        "3-Day Rent: Rs. " + v.calculateRent(3));
                System.out.println();
            }

            Vehicle.showVehicleCount();

            // Constructor overloading
            Customer customer =
                    new Customer(501, "Kajal", "9876543210");

            System.out.println("\n===== CUSTOMER =====");
            System.out.println(customer);

            // Association + interface + exception handling
            Rental rental =
                    new Rental(customer, vehicles[0]);

            double amount = rental.calculateTotal(3, 10);

            System.out.println("\n===== RENTAL =====");
            System.out.println("Customer: " + customer.getName());
            System.out.println("Vehicle: " + vehicles[0].getBrand());
            System.out.println("Final Amount: Rs. " + amount);

            rental.pay(amount);

            Rental.showRentalCount();

        } catch (InvalidVehicleException e) {

            System.out.println("Vehicle Error: " + e.getMessage());

        } catch (InvalidCustomerException e) {

            System.out.println("Customer Error: " + e.getMessage());

        } catch (VehicleNotAvailableException e) {

            System.out.println("Rental Error: " + e.getMessage());

        } catch (IllegalArgumentException e) {

            System.out.println("Input Error: " + e.getMessage());

        } finally {

            System.out.println("\nRental operation completed.");
        }
    }
}
```

How the Solution Demonstrates the Concepts

Concept	Where Demonstrated
Encapsulation	Private fields + getters/setters.
Inheritance	Car, Bike and ElectricCar extend Vehicle.
Abstraction	Vehicle is abstract and calculateRent() is abstract.
this	Vehicle, Customer and Rental constructors.
super	Child constructors call the Vehicle constructor.
Constructor overloading	Customer has three constructors.
Method overloading	Rental.calculateTotal() has three versions.
Method overriding	Each vehicle overrides calculateRent().
Runtime polymorphism	Vehicle[] stores Car/Bike/ElectricCar objects.
Interface	Rental implements Payment.
static	Vehicle and Rental counters.
final	GST and displayBasicDetails().
Association	Rental contains Customer and Vehicle references.
Arrays of objects	Vehicle[] in main().
throw	Validation creates custom exceptions.
throws	Constructors and rental methods declare exceptions.
try/catch/finally	main() handles errors and cleanup message.
Custom exceptions	Three domain-specific exception classes.

Suggested 2-Hour Teaching / Lab Flow

0–20 min: Vehicle, encapsulation and constructors.

20–45 min: Car/Bike/ElectricCar, inheritance, super and overriding.

45–65 min: Customer, constructor overloading and this.

65–85 min: Payment, Rental and method overloading.

85–105 min: Runtime polymorphism using Vehicle[].

105–120 min: Custom exceptions, throw/throws, try-catch-finally and testing.